CPC — Lecture 10 (provisional)

Applications of prefix sums · Fenwick tree (start)

Competitive Programming and Contests — Monday 19/10/2026

PROVISIONAL — recording not available yet. Reconstructed from your notes taken during the lecture (`Lessons/L10/notes.md`) and from the prof's blog posts *The Power of Prefix Sums* (`https://pages.di.unipi.it/rossano/blog/2023/prefixsums`) and *Dynamic Prefix Sums with Fenwick Tree* (`https://pages.di.unipi.it/rossano/blog/2023/fenwick`), whose code is copied verbatim. What comes from the blog and not from your notes is marked **[blog]**; my additions are marked [extra]. To be redone on the recording when it is out.

Code: `Lessons/L10/src` (`cargo test -p l10`).
Highlights: general / to remember good: works, efficient, correct bad: pitfalls, wrong ideas, costs definitions and key results
Where we are. L9 (14/10) ended Frogs and Mosquitoes and started static prefix sums with *Ilya and Queries*; the prof announced the other prefix-sum problems and then dynamic prefix sums for this lecture.

Contents

1 Little Girl and Maximum Sum (Codeforces 276C) 1

2 Number of Ways (Codeforces 466C) 2

3 Fenwick tree (Binary Indexed Tree) 3
3.1 Construction (from your notes) 4

4 Next time 5

1 Little Girl and Maximum Sum (Codeforces 276C)

Problem. An array $A[1, n]$ and a set Q of m queries, each a RangeSum(i, j). Permute the values of A to maximize the total of the answers of the queries.
Example (your notes). $A = [5, 2, 4, 1, 3]$, total 15; with the queries $(1, 5), (2, 3), (2, 3)$ the value is $15 + (2 + 4) + (2 + 4) = 27$; with $A_\pi = [2, 5, 4, 1, 3]$ it is $15 + (5 + 4) + (5 + 4) = 33$. [extra] Your notes write the queries as $(1, 5), (1, 2), (1, 2)$, but the sums $2+4$ and the frequencies $1, 3, 3, 1, 1$ correspond to $(2, 3)$: I used $(2, 3)$.
Idea. Put the largest values in the positions touched by most queries.

1. **Count the frequency of every position** (how many queries contain it). Example: $F = [1, 3, 3, 1, 1]$; assigning $5, 4$ to the two positions of frequency 3 gives 33.
   - with a sweep line (your notes: $\Theta(n \log m)$);
   - with prefix sums: build an array B whose prefix sums are F: every query (i, j) adds $+1$ at i and -1 at $j + 1$ ("the position after the end of a segment contributes -1"). Example: $Q = (2, 4), (1, 3), (2, 5) \Rightarrow B = [1, 2, 0, -1, -1]$, prefix sums $F = [1, 3, 3, 2, 1]$. $\Theta(n + m)$. Equivalently two counting arrays, $C_{\text{begins}} = [1, 2, 0, 0, 0]$ and $C_{\text{ends}} = [0, 0, 1, 1, 1]$: B is their combination.

2. **Sort** A in decreasing order and the positions by decreasing frequency, and assign the largest value to the most frequent position.

Your notes give the total time as $\Theta(n \log n + m \log m)$. [extra] With the B array the frequencies cost $\Theta(n + m)$ and only the two sorts of n values remain: $\Theta(n \log n + m)$ (the $m \log m$ belongs to a version that sorts the queries).

Prof's code [blog] (verbatim):

```rust
// We assumes queries are 0-based indexed
pub fn little_girl(a: &[i64], q: &[(usize, usize)]) -> i64 {
    if a.is_empty() {
        return 0;
    }

    let mut u = vec![0i64; a.len()];

    for &(l, r) in q {
        assert!(l <= r);
        assert!(r < u.len());

        u[l] += 1;
        if r + 1 < u.len() {
            u[r + 1] -= 1;
        }
    }

    let mut f = u
        .iter()
        .scan(0, |sum, e| {
            *sum += e;
            Some(*sum)
        })
        .collect::<Vec<_>>();

    // we sort both f and a in decreasing order, nothing changes
    f.sort_unstable();
    let mut a_sorted = a.to_vec();
    a_sorted.sort_unstable();

    a_sorted
        .iter()
        .zip(f)
        .fold(0, |result, (value, freq)| result + value * freq)
}
```

[extra] It sorts both in *increasing* order: pairing the k-th smallest with the k-th smallest is the same as pairing the largest with the largest.

2 Number of Ways (Codeforces 466C)

(*"Problem #3" in your notes; problem #2 is not in your notes.*)

Problem. Given $A[1, n]$, count in how many ways we can split A into 3 (non-empty, contiguous) parts with the same sum. Example: $A = [1, 2, 3, 0, 3]$ (answer 2: $[1, 2 \mid 3 \mid 0, 3]$ and $[1, 2 \mid 3, 0 \mid 3]$ [extra]).

Idea (your notes). S = sum of A, target $T = S/3$ (if $3 \nmid S$ the answer is 0 [extra]).

- Scan left to right computing prefix sums and mark with "+" every position where the prefix sum is T (end of a valid first part).
- Scan right to left and mark "red" every position where the suffix sum is T (start of a valid third part).
- Answer = number of pairs ("+", red) with the red point to the right of the "+" (leaving a non-empty middle part). For each "+", the number of red points to its right is a suffix sum on the red marks: precompute it and the whole thing is linear.

[extra] Why the middle part is fine automatically: if the first part sums to T and the last to T, the middle sums to $S - 2T = T$.

Prof's code [blog] (verbatim):

```rust
pub fn number_of_ways(a: &[i64]) -> usize {
    let sum: i64 = a.iter().sum();

    if sum % 3 != 0 {
        return 0;
    }

    let target = sum / 3;
    let mut c: Vec<_> = a
        .iter()
        .rev()
        .scan(0, |sum, e| {
            *sum += e;
            Some(*sum)
        })
        .scan(0, |counter, sum| {
            if sum == target {
                *counter += 1usize
            };
            Some(*counter)
        })
        .collect();

    c.reverse();

    let mut result = 0;
    let mut sum = 0;
    for (i, &v) in a[..a.len() - 2].iter().enumerate() {
        sum += v;
        if sum == target {
            result += c[i + 2];
        }
    }

    result
}
```

[extra] $c[i]$ = number of red points at positions $\geq i$ (the suffix sum on the red marks); `c[i + 2]` leaves at least one element in the middle. Pitfall: with $n = 1$ (allowed by Codeforces) `a.len() - 2` underflows and the function panics (`number_of_ways(&[0])`); guard with `if a.len() < 3 { return 0; }`. Tested in `l10` against an $O(n^2)$ brute force.

3 Fenwick tree (Binary Indexed Tree)

Problem: dynamic prefix sums. Given $A[1, n]$, support $\mathsf{Sum}(i) = \sum_{k=1}^{i} A[k]$ and $\mathsf{Add}(i, v)$: $A[i]$ += v (and $\mathsf{RangeSum}(i, j) = \mathsf{Sum}(j) - \mathsf{Sum}(i-1)$).
Trivial solutions.

- Store A: Sum $\Theta(n)$, Add $O(1)$.
- Store the prefix sums: Sum $O(1)$, Add $O(n)$ (all the following prefix sums change).

The BIT supports both in $\Theta(\log n)$ time.

3.1 Construction (from your notes)

Example: $A = [3, 2, -1, 5, 7, -3, 2, 1]$, $n = 8$.

1. **Relaxed problem**: answer $\mathsf{Sum}(i)$ only when i is a power of 2 (and Add for any i). Store only those prefix sums: $[3, 5, \cdot, 9, \cdot, \cdot, \cdot, 16]$. Sum $O(1)$, Add $\Theta(\log n)$ (only $\log n$ stored values can change).
2. Add a fake root at position 0 (value 0) and link to it the stored nodes, which cover the ranges $[1, 1], [1, 2], [1, 4], [1, 8]$: positions $0001, 0010, 0100, 1000$ in binary, exactly one bit set.
3. Cover the uncovered parts by applying the same principle recursively. Between 4 and 8 the uncovered part is $[7, -3, 2]$ (positions $5, 6, 7$, i.e. $1, 2, 3$ shifted by 4): its relaxed solution stores $[5, 5] = 7$ and $[5, 6] = 4$, positions 0101 and 0110 (two bits set), children of node 4. Then $[-1]$ at position $3 = 0011$, range $[3, 3]$, child of node 2; and 2 at position $7 = 0111$ [extra].
4. Observation in your notes: the number of trailing 0s in the binary representation of a node = number of its children.

Operations (your notes stop here, running times not written):

- $\mathsf{Sum}(i)$: start from the node whose range ends at i (node i) and sum its value and the values of its ancestors up to the root.
- $\mathsf{Add}(i, v)$: update the values of all the nodes whose range contains i.

[extra] Node i covers the range $(i - \mathsf{lsb}(i), i]$, where $\mathsf{lsb}(i)$ is the lowest set bit of i. The parent of i is $i - \mathsf{lsb}(i)$ (clear the lowest set bit), so Sum visits at most $\log n + 1$ nodes; Add visits i, $i + \mathsf{lsb}(i), \ldots$ (the "next sibling" in the blog), again $O(\log n)$ nodes. For the example: the array is $[0 \mid 3, 5, -1, 9, 7, 4, 2, 16]$; $\mathsf{Sum}(7) = T[7] + T[6] + T[4] = 2 + 4 + 9 = 15$.

Prof's code [blog] (verbatim):

```rust
#[derive(Debug)]
pub struct FenwickTree {
    tree: Vec<i64>,
}

impl FenwickTree {
    pub fn with_len(n: usize) -> Self {
        Self {
            tree: vec![0; n + 1],
        }
    }

    pub fn len(&self) -> usize {
        self.tree.len() - 1
    }

    /// Indexing is 0-based, even if internally we use 1-based indexing
    pub fn add(&mut self, i: usize, delta: i64) {
        let mut i = i + 1;
        assert!(i < self.tree.len());
```

```rust

        while i < self.tree.len() {
            self.tree[i] += delta;
            i = Self::next_sibling(i);
        }
    }

    /// Indexing is 0-based, even if internally we use 1-based indexing
    pub fn sum(&self, i: usize) -> i64 {
        let mut i = i + 1;

        assert!(i < self.tree.len());
        let mut sum = 0;
        while i != 0 {
            sum += self.tree[i];
            i = Self::parent(i);
        }

        sum
    }

    pub fn range_sum(&self, l: usize, r: usize) -> i64 {
        self.sum(r) - if l == 0 { 0 } else { self.sum(l - 1) }
    }

    fn isolate_trailing_one(i: usize) -> usize {
        if i == 0 {
            0
        } else {
            1 << i.trailing_zeros()
        }
    }

    fn parent(i: usize) -> usize {
        i - Self::isolate_trailing_one(i)
    }

    fn next_sibling(i: usize) -> usize {
        i + Self::isolate_trailing_one(i)
    }
}
```

[extra] Tested in `l10::fenwick`: on the example (internal array $[0, 3, 5, -1, 9, 7, 4, 2, 16]$) and against the trivial array on random add/sum sequences.

4 Next time

Probably the end of the Fenwick tree (running times, why it works) and its applications. [extra] In last year's course (course page): counting inversions, *Update the array* (SPOJ UPDATEIT, mandatory in 2025), range updates, *Nested segments* (Codeforces 652D). To be checked on the recording.